Artificial Intelligence Internship

Skillify



Task 4 Report

AI Ready Dataset Preperation

Submitted by

Nayab Maryam

Submitted to

Mam Momina Arshad

Task 3: AI Ready Dataset Preperation:

Objective

Prepare a raw dataset for machine learning through cleaning, preprocessing, visualization, and feature transformation.

Requirements

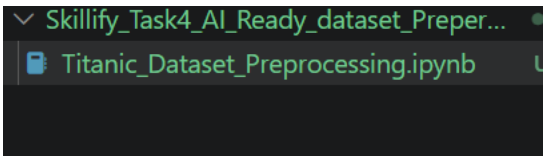
- Select a suitable dataset.
- Handle missing values, duplicates, and outliers.
- Encode categorical features.
- Scale numerical features.
- Create at least 3 visualizations.
- Export the final ML-ready dataset.

Submission format

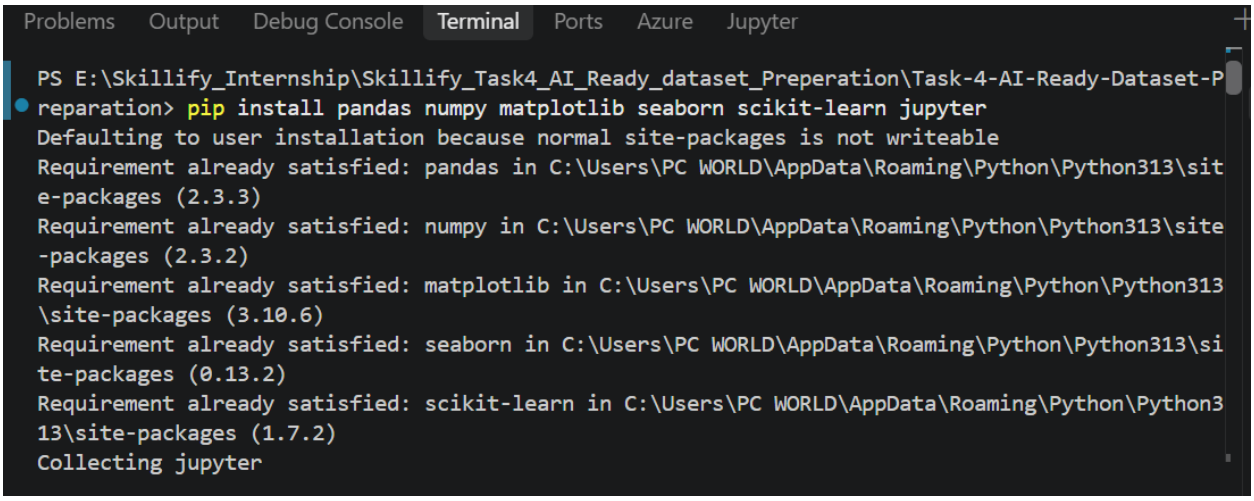
- Python notebook/file (.ipynb / .py)
- Visualizations/screenshots

Solution:

First I build the basic structure of my task:



I installed the libararies:



I imported them into my notebook

Then I used Seaborn's built-in Titanic dataset. It contains real passenger information and is commonly used for learning data analysis and preprocessing.

For it I used this command df = sns.load_dataset('titanic')

And then to explore this dataset first I used head to show first few rows and then shap to know the dimensions of Dataset.

```
df.head()
```
✓ 0.1s Python

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Southampton

```
### Check Dataset Dimensions:
```
markdown

```
df.shape
```
✓ 0.0s Python

```
(891, 15)
```

After exploring the dataset now I am going for quality check of dataset

I first check for missing values: and the output I got is

```
survived          0
pclass            0
sex               0
age             177
sibsp             0
parch             0
fare              0
embarked          2
class             0
who               0
adult_male        0
deck            688
embark_town       2
alive             0
alone             0
dtype: int64
```

```
survived        0.00
pclass          0.00
sex             0.00
age            19.87
sibsp           0.00
parch           0.00
fare            0.00
embarked        0.22
class           0.00
who             0.00
adult_male      0.00
deck           77.22
embark_town     0.22
alive           0.00
alone           0.00
dtype: float64
```

And this the anlayiss of result and how we resolve it

Column	Missing values	Percentage	Initial decision
age	177	19.87%	Fill with median
embarked	2	0.22%	Fill with mode
deck	688	77.22%	Consider dropping
embark_town	2	0.22%	Fill or remove redundant column

Why these decisions?

- `age` : Almost 20% is missing. We'll use the median because age can contain outliers, and the median is less affected by extreme values than the mean.
- `embarked` : Only two values are missing. We can replace them with the most frequent category.
- `deck` : More than 77% is missing. Imputing this column could introduce too much unreliable information. We'll probably remove it.
- `embark_town` : It contains the same information as `embarked` , so we'll eventually retain only one.

Then I check for duplicate records

```
# Count duplicate rows
duplicate_count = df.duplicated().sum()

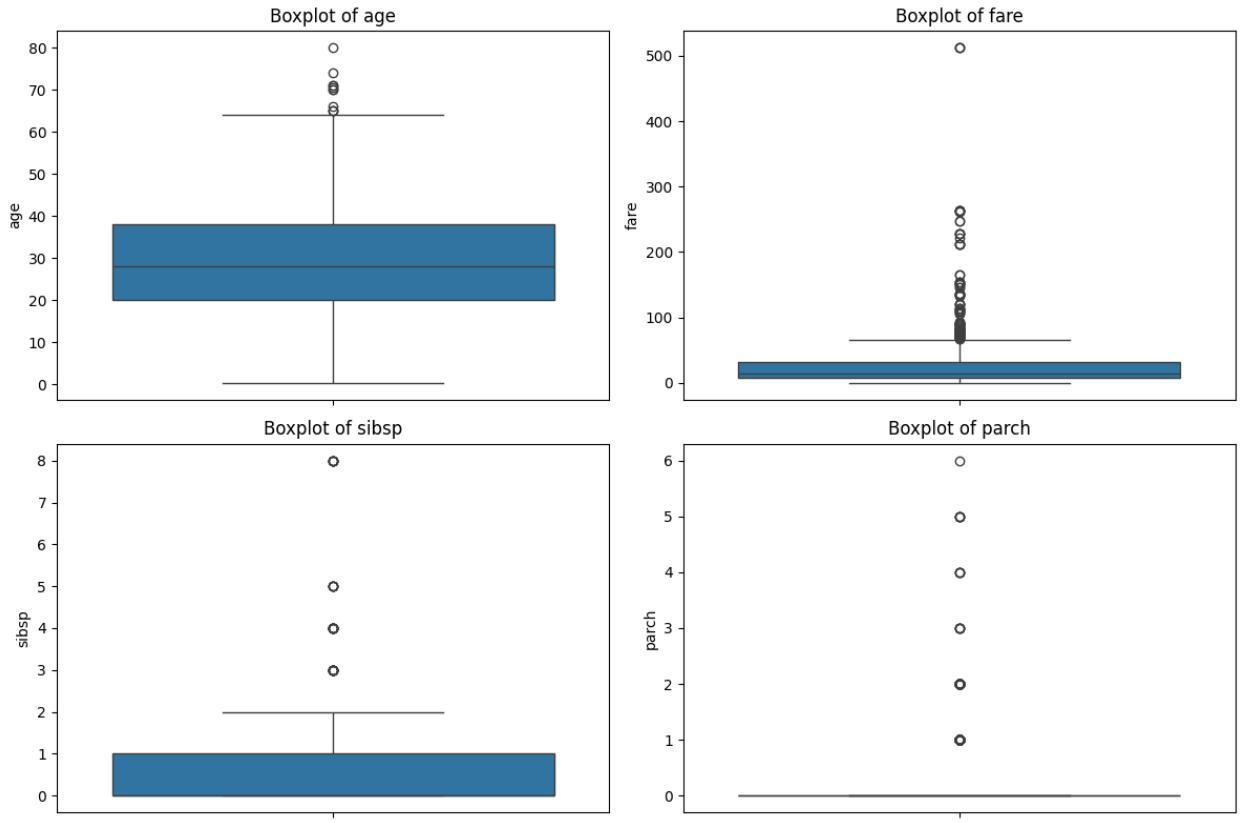
print("Total duplicate rows:", duplicate_count)
```

✓ 0.0s

```
Total duplicate rows: 107
```

And we got 107 records are duplicates

And then we use box plot to find outliers



And we observed the following things:

Here are my observations:

Feature	Observation	Decision
age	Some unusually high ages	Investigate, likely retain
fare	Many high-value outliers	Investigate, likely retain
sibsp	Some large family counts	Retain unless invalid
parch	Several unusual family counts	Retain unless invalid

To address the issue of duplicates to avoid accidently deleting the records that isnot actually duplicate we will inspect it to do that I first display the duplicated records and then check for duplicate groups

```
Total rows involved in duplicates: 160
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	em
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Sc
26	0	3	male	NaN	0	0	7.2250	C	Third	man	True	NaN	
28	1	3	female	NaN	0	0	7.8792	Q	Third	woman	False	NaN	C
29	0	3	male	NaN	0	0	7.8958	S	Third	man	True	NaN	Sc
32	1	3	female	NaN	0	0	7.7500	Q	Third	woman	False	NaN	C
37	0	3	male	21.0	0	0	8.0500	S	Third	man	True	NaN	Sc
42	0	3	male	NaN	0	0	7.8958	C	Third	man	True	NaN	
45	0	3	male	NaN	0	0	8.0500	S	Third	man	True	NaN	Sc
46	0	3	male	NaN	1	0	15.5000	Q	Third	man	True	NaN	C
47	1	3	female	NaN	0	0	7.7500	Q	Third	woman	False	NaN	C

```python
duplicate_rows = df[df.duplicated(keep=False)]

# Count unique duplicated records
unique_duplicate_records = duplicate_rows.drop_duplicates()

print("Total rows involved in duplicate groups:", len(duplicate_rows))

print("Number of duplicate groups:", len(unique_duplicate_records))

# Display first 10 duplicate records
display(duplicate_rows.head(10))
```

✓ 0.0s Python

```
Total rows involved in duplicate groups: 160
Number of duplicate groups: 53
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	em
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Sc
26	0	3	male	NaN	0	0	7.2250	C	Third	man	True	NaN	

Now for cleaning the dataset phase :

I first remove unnecessary columns from and build the new table with only needed columns:

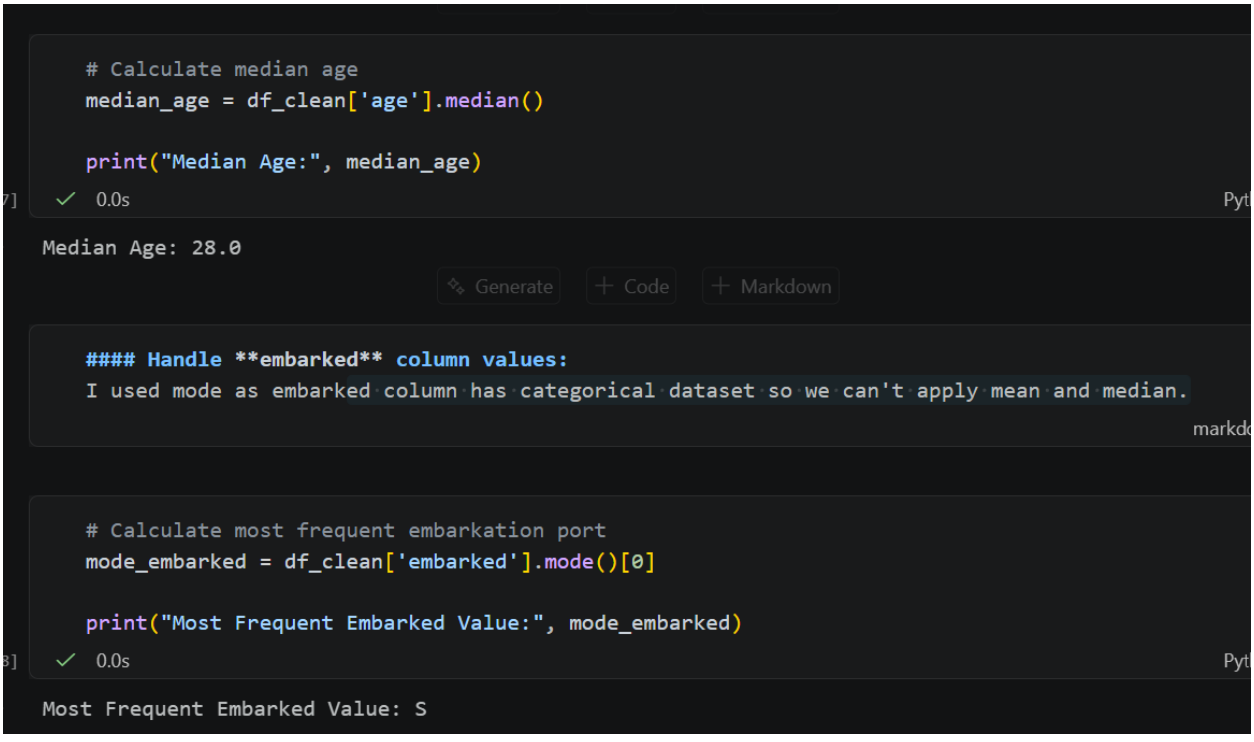
```
Original dataset shape: (891, 15)
Selected dataset shape: (891, 8)
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked
0	0	3	male	22.0	1	0	7.2500	S
1	1	1	female	38.0	1	0	71.2833	C
2	1	3	female	26.0	0	0	7.9250	S
3	1	1	female	35.0	1	0	53.1000	S
4	0	3	male	35.0	0	0	8.0500	S

Now I will handle missing records

First I handle age column values I used median as it better handle outliers and large and low values in dataset as age column ahs this

And then to handle embarked column I used mode as this column has categorical values so we can’t use mean and median.



No we will do outliers detection and treatment.

First discuss about outliers values that are the values unusually different from most other observations

Interquartile Range (IQR) method.

The formula is:

IQR = Q3 - Q1

Where:

- Q1 = 25th percentile.
- Q3 = 75th percentile.

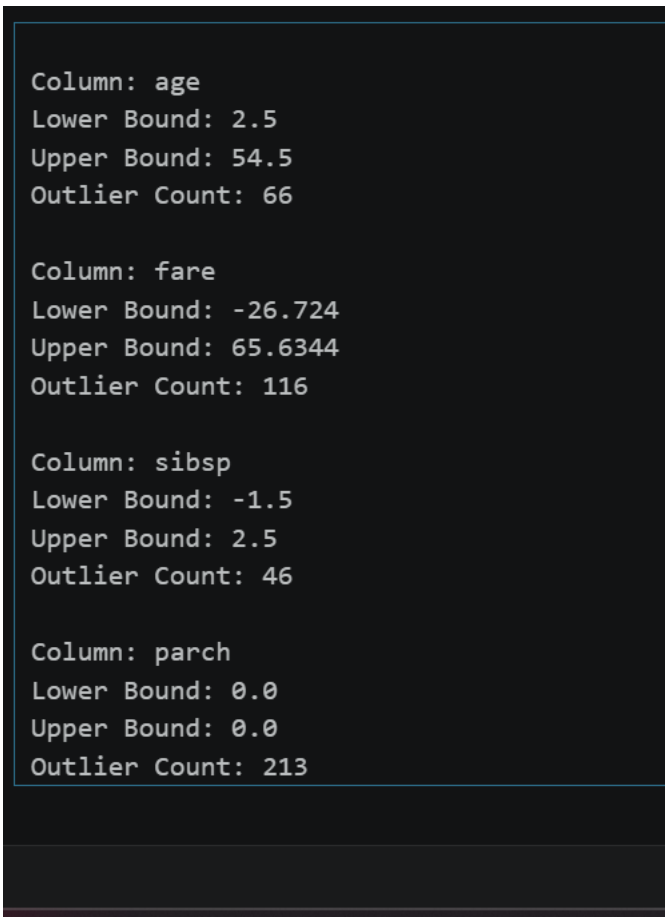
We calculate the boundaries:

Lower = Q1 - 1.5(IQR)

Upper = Q3 + 1.5(IQR)

Values outside these boundaries are potential outliers.

Now we clclaute IQR na detect no of outliers in each column



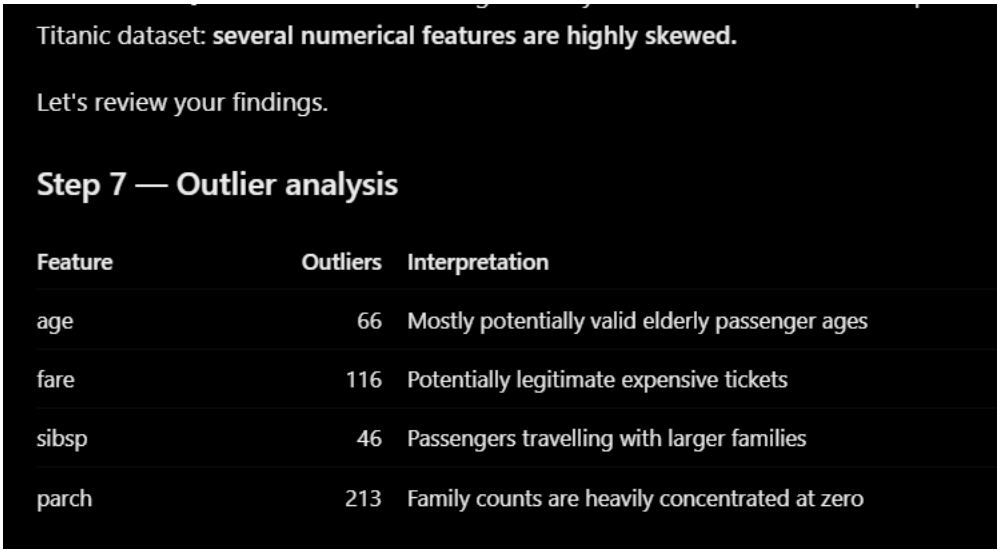
```
Column: age
Lower Bound: 2.5
Upper Bound: 54.5
Outlier Count: 66

Column: fare
Lower Bound: -26.724
Upper Bound: 65.6344
Outlier Count: 116

Column: sibsp
Lower Bound: -1.5
Upper Bound: 2.5
Outlier Count: 46

Column: parch
Lower Bound: 0.0
Upper Bound: 0.0
Outlier Count: 213
```

The observation is



Titanic dataset: **several numerical features are highly skewed.**

Let's review your findings.

Step 7 — Outlier analysis

Feature	Outliers	Interpretation
age	66	Mostly potentially valid elderly passenger ages
fare	116	Potentially legitimate expensive tickets
sibsp	46	Passengers travelling with larger families
parch	213	Family counts are heavily concentrated at zero

our upper boundary is 0.0.

Why?

Because the majority of passengers had zero parents or children aboard. Both Q1 and Q3 are zero, making the IQR zero.

Consequently, every positive value is flagged as an outlier.

(Iska matlab ye nahi ke 213 records incorrect hain. Sirf itna hai ke data mein zero values bohat zyada hain aur IQR method is feature ke liye suitable result nahi de raha)

The outliers treatment plan is below :

2. Our outlier treatment strategy

I don't recommend blindly deleting these records.

Instead, we'll use the following approach:

Feature	Our decision	Reason
age	Retain	Elderly passengers are legitimate
fare	Apply log transformation	Reduce extreme right-skewness
sibsp	Retain	Large families are valid
parch	Retain	IQR is unsuitable for this zero-heavy feature

For `fare`, we'll use `np.log1p()` later.

This transforms values using:

$$X_{new} = \log(1 + X)$$

It reduces the influence of very large values while preserving the ordering of fares.

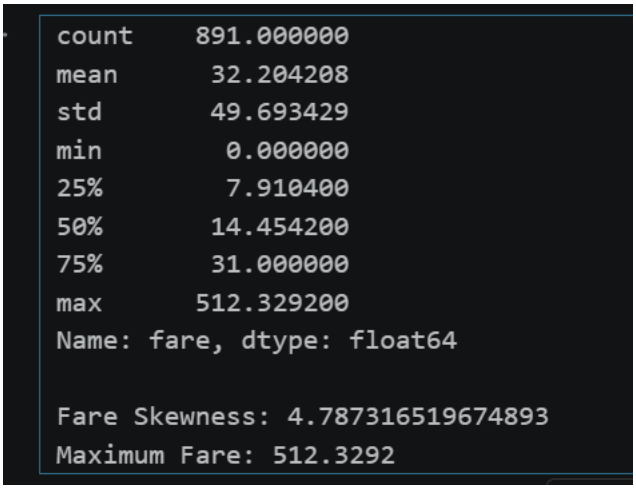
Before applying the transformation on fare we will se is fare actually skewed

What skewed tells us :

- Skewness close to 0: approximately symmetric distribution.

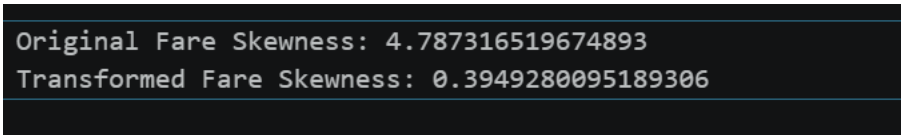
- Positive skewness: long tail towards higher values.
- Negative skewness: long tail towards lower values.

So the results we got are:

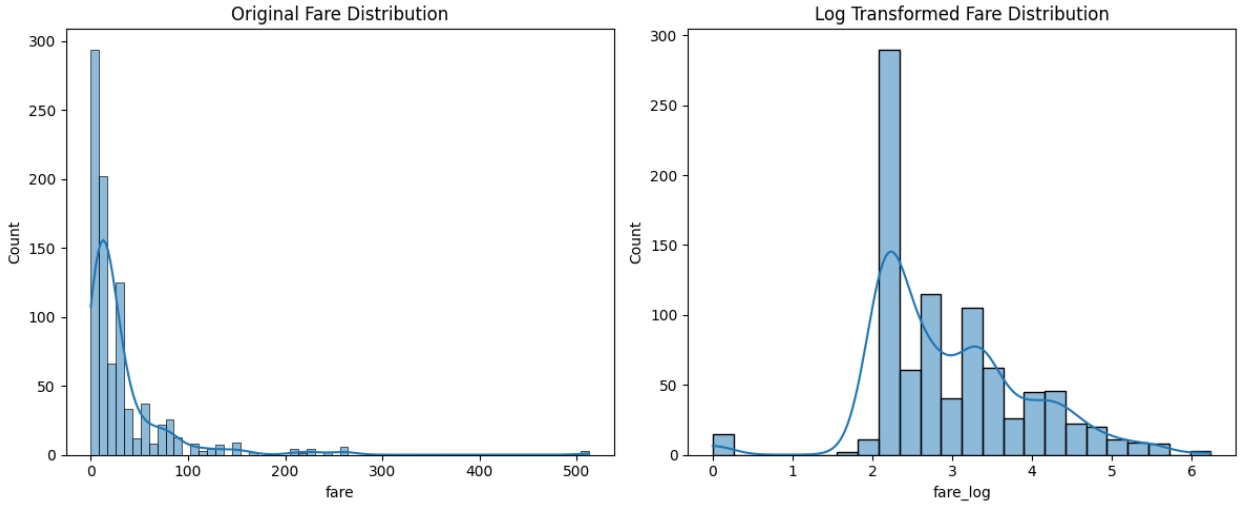


As u can see fare skwedness is highly positive so here log transformation is useful now we will apply it.

And this is the result we got:



We use matplotlib to plot this graph to show the clear difference of original fare distribution and after transformation fare distribution.



Now as u see the transformed fare is better so instead of using original fare we added fare_log as new cfeature(column) in our table

```
['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'embarked', 'fare_log']
```

	survived	pclass	sex	age	sibsp	parch	embarked	fare_log
0	0	3	male	22.0	1	0	S	2.110213
1	1	1	female	38.0	1	0	C	4.280593
2	1	3	female	26.0	0	0	S	2.188856
3	1	1	female	35.0	1	0	S	3.990834
4	0	3	male	35.0	0	0	S	2.202765

Now it time to apply categorical Encoding as out data of sex,embarked ,pclass is categorical and machine learning requires data to be in numerical input so we will convert this columns to numerical representations

We'll use One-Hot Encoding.

For example, the `sex` column:

Original	sex_female	sex_male
female	1	0
male	0	1

Similarly, `embarked` will become three separate columns.

After applying these are the result:



g	pclass_1	pclass_2	pclass_3	sex_female	sex_male	embarked_C	embarked_Q	embarked_S
3	0	0	1	0	1	0	0	1
3	1	0	0	1	0	1	0	0
6	0	0	1	1	0	0	0	1
4	1	0	0	1	0	0	0	1
5	0	0	1	0	1	0	0	1

So the new columns are:

Updated Dataset Shape: (891, 13)

Updated Columns:

['survived', 'age', 'sibsp', 'parch', 'fare_log', 'pclass_1', 'pclass_2', 'pclass_3', 'sex_female', 'sex_male', 'embarked_C', 'embarked_Q', 'embarked_S']

Remaining Categorical Columns:

[]

Before doing feature scaling we first split dataset into tst and train and the results are:

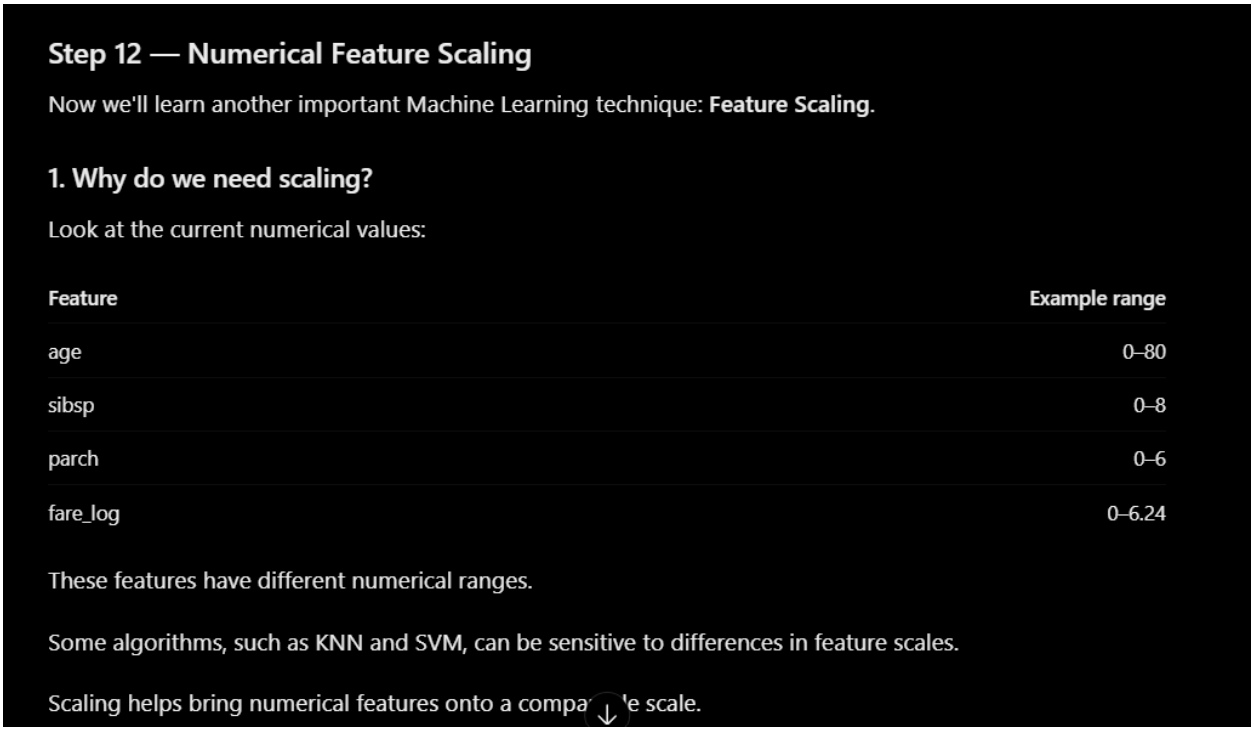
Training features: (712, 12)

Testing features: (179, 12)

Training target: (712,)

Testing target: (179,)

Now we will apply another technique of machine learning that is feature scaling:



Step 12 — Numerical Feature Scaling

Now we'll learn another important Machine Learning technique: **Feature Scaling**.

1. Why do we need scaling?

Look at the current numerical values:

Feature	Example range
age	0–80
sibsp	0–8
parch	0–6
fare_log	0–6.24

These features have different numerical ranges.

Some algorithms, such as KNN and SVM, can be sensitive to differences in feature scales.

Scaling helps bring numerical features onto a compa e scale.

We'll use `StandardScaler` from Scikit-learn.

Its formula is:

$$Z = \frac{X - \mu}{\sigma}$$

Where:

- X = Original value
- μ = Mean
- σ = Standard deviation

After scaling, each feature will approximately have:

- Mean = 0
- Standard deviation = 1

Roman Urdu: StandardScaler har numerical feature ka mean zero aur standard deviation one ke qareeb kar deta hai. Is se different ranges wale features compara scale par aa jate hain.

We applied standard scale

These are the results:

```
Training data means:
age         0.0
sibsp      -0.0
parch      -0.0
fare_log    0.0
dtype: float64

Training data standard deviations:
age         1.0
sibsp       1.0
parch       1.0
fare_log    1.0
dtype: float64
```

```
print("Testing data sample:")
display(X_test.head())
```

✓ 0.0s Python

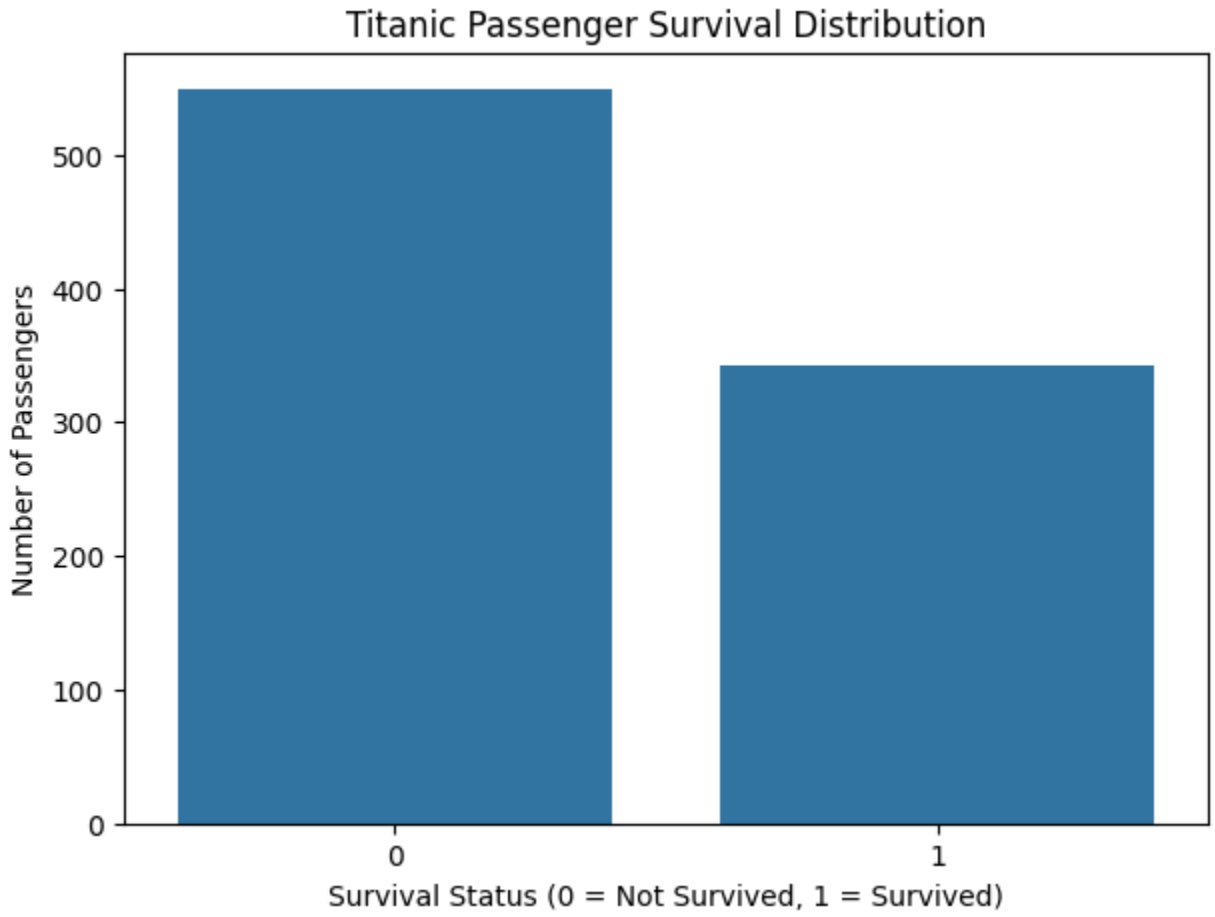
```
Testing data sample:
```

	age	sibsp	parch	fare_log	pclass_1	pclass_2	pclass_3	sex_female	sex_male	embarked_C	emba
565	-0.419170	1.421753	-0.466183	0.279952	0	0	1	0	1	0	
160	1.116290	-0.465084	0.727782	-0.114127	0	0	1	0	1	0	
553	-0.572716	-0.465084	-0.466183	-0.861771	0	0	1	0	1	1	
860	0.885971	1.421753	-0.466183	-0.240625	0	0	1	0	1	0	

And now its time for data visaulizations:

Visualization 1: Survival distribution

Our first visualization will show how many passengers survived and how many did not.



Concept (Roman Urdu): Is graph se humein pata chalega ke total passengers mein se kitne survive hue aur kitne nahi hue.

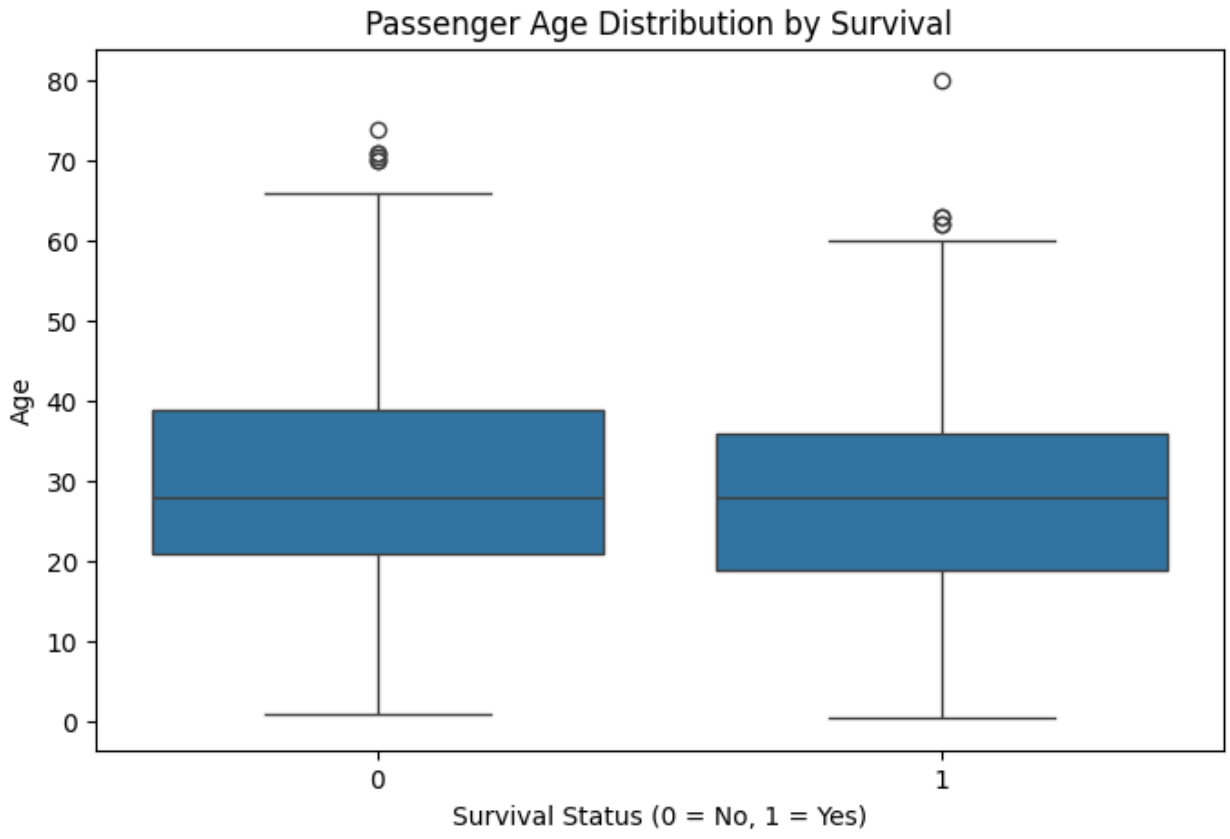
Your graph shows:

- 549 passengers did not survive.
- 342 passengers survived.
- More passengers did not survive than survived.

Finding: Approximately 38.4% of passengers survived.

Visualization 2: Age distribution by survival

We'll compare the ages of passengers who survived and those who didn't.



Concept: Is graph se hum compare kar sakte hain ke different age groups mein survival ka pattern kaisa tha.

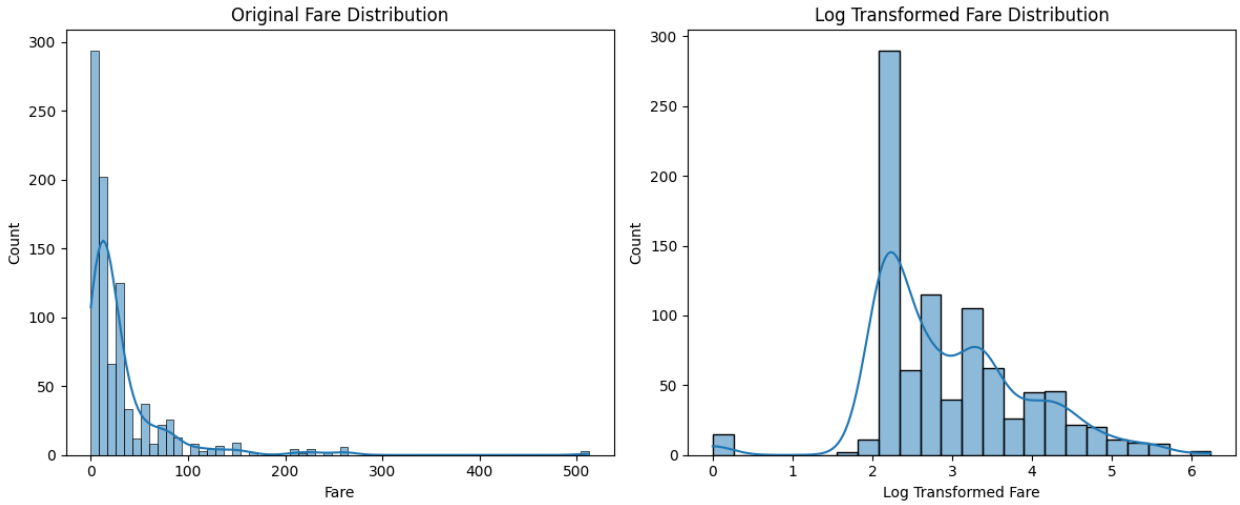
Your boxplot shows:

- The median age for both groups is approximately 28 years.
- Both groups contain passengers from a wide range of ages.
- Some elderly passengers appear as potential outliers.

Finding: The median ages of survivors and non-survivors are similar. Age alone does not explain survival outcomes.

Visualization 3: Fare distribution before and after transformation

This graph will demonstrate the improvement achieved through log transformation.



Concept: Ye visualization clearly show karegi ke log transformation se fare ki highly skewed distribution kis tarah improve hui.

After that I coded some finalized dataset validations like count missing values,duplicated values,datatypes ,dimensions etc and then exported the csv

